Conflict-Free Replicated Data Types for Neural Network Model Merging: A Two-Layer Architecture Enabling CRDT-Compliant Model Merging Across 26 Strategies

Ryan Gillespie
crdt-merge

Abstract

All 26 neural network merge strategies we tested—including weight averaging, SLERP, TIES, DARE, Fisher merging, and evolutionary approaches—fail the algebraic properties (commutativity, associativity, idempotency) required for conflict-free distributed operation [29]. We prove that this failure is structural: normalisation-based merges *cannot* simultaneously satisfy all three properties. To resolve this, we present a two-layer architecture—CRDTMergeState—that wraps *any* merge strategy in a CRDT-compliant (Conflict-Free Replicated Data Type) layer. Layer 1 manages contributions via OR-Set CRDT semantics [28], where the merge operation is set union—trivially commutative, associative, and idempotent. Layer 2 applies merge strategies as deterministic pure functions over a canonically-ordered contribution set, with randomness seeded from the Merkle root [26]. We prove that this separation guarantees Strong Eventual Consistency [29]: all replicas receiving the same contributions compute identical merged models, regardless of message ordering. Empirical validation spans three tiers: controlled 4×4 tensors (104/104 tests pass), production-scale models up to 7.24 B parameters (208 strategy-level tests, 43,368 layer-level property checks at capped tensor resolution), and multi-node convergence under gossip and partition healing (100 nodes, 20 orderings), with CRDT overhead below 0.5 ms. Because the wrapper is transparent, downstream performance is identical by construction, confirmed via byte-identical output verification. The reference implementation is available as `crdt-merge` v0.9.4.

1 Introduction

As large-scale neural network models multiply, methods for combining independently fine-tuned models without retraining are essential [35, 32]. Model merging—combining the parameters of two or more neural networks into a single model—offers a practical alternative to ensemble methods and multi-task training [32, 12], with strategies ranging from weight averaging [32] and Task Arithmetic [12] to TIES [33], DARE [37], Fisher merging [22], SLERP [30], and evolutionary methods [1], supported by tools such as MergeKit [10].

Despite this progress, *no existing merge strategy satisfies the algebraic properties required for conflict-free distributed operation.* Conflict-Free Replicated Data Types (CRDTs) [29, 28] guarantee Strong Eventual Consistency (SEC) by requiring merge operations to be commutative, associative, and idempotent [24, 31]. As we demonstrate in Section 3, all 26 strategies fail at least one property, with associativity as the universal failure point (25/26 fail). This prevents decentralised model merging—where participants combine models peer-to-peer without a central coordinator—a capability relevant to multi-institutional collaboration (e.g., research consortia) where participants prefer not to rely on a single aggregation server [15, 5]; adversarial settings additionally require Byzantine fault tolerance (Section 7.2, L4).

Contributions.

1. A systematic algebraic audit of 26 neural network merge strategies revealing *universal associativity failure* (25/26 strategies), together with a formal result (Proposition 4) proving that normalisation-based merges cannot satisfy all CRDT axioms simultaneously (Section 3).

2. A two-layer architecture—CRDTMergeState—achieving universal CRDT compliance by separating state management (Layer 1, OR-Set semantics) from strategy execution (Layer 2, deterministic pure functions). Applying CRDTs directly to merge operations is impossible (Section 3); the two-layer separation is the key architectural insight. While CRDT composition is a known pattern [28], the domain-specific challenges—stochastic strategies requiring Merkle-root-derived seeding, order-dependent reductions requiring canonical hashing, and high-dimensional floating-point determinism—required careful engineering detailed in Section 4.

3. Formal proofs that the architecture guarantees Strong Eventual Consistency for arbitrary merge strategies (Section 5), with explicit complexity bounds (Theorem 15).

4. Empirical validation at three tiers: controlled 4×4 tensors (104/104 tests), production-scale models up to 7.24 B parameters (208/208 strategy-level tests, 43,368 layer-level evaluations), and multi-node convergence under gossip and partition healing, with CRDT overhead below 0.5 ms (Section 6).

2 Background

2.1 Conflict-Free Replicated Data Types

Conflict-Free Replicated Data Types (CRDTs) are data structures for replicated settings where concurrent updates must be merged without coordination [29, 28]. They guarantee Strong Eventual Consistency (SEC): any two replicas that have received the same set of updates converge to identical states [31], as exemplified by Amazon's Dynamo [7].

Definition 1 (State-based CRDT / CvRDT [29])**.** *A convergent replicated data type (CvRDT) is a tuple (S, s_0, q, u, m) where S is a join-semilattice of states with partial order $\leq$, s_0 is the initial state, q is a query function, u is an update function, and $m : S \times S \to S$ is a merge function satisfying:*

$$m(s_1, s_2) = m(s_2, s_1) \qquad \textit{(Comm.)} \quad (1)$$

$$m(m(s_1, s_2), s_3) = m(s_1, m(s_2, s_3)) \qquad \textit{(Assoc.)} \quad (2)$$

$$m(s, s) = s \qquad \textit{(Idemp.)} \quad (3)$$

These three laws ensure that the merge operation forms a join (least upper bound) on the semilattice, guaranteeing convergence regardless of message ordering or duplication [28, 24].

Definition 2 (OR-Set [29])**.** *An Observed-Remove Set (OR-Set) is a CRDT that supports both add and remove operations. Each element is tagged with a unique identifier upon insertion. A remove operation removes all observed tags for an element, allowing concurrent adds to survive. The merge operation is set union over the tagged elements minus the tombstoned tags.*

In the model merging context, an *add* represents a participant contributing a fine-tuned model, while a *remove* represents retraction. Under OR-Set "add-wins" semantics, a concurrent add survives a concurrent remove—a natural default for collaborative model development where contributions should be preserved unless explicitly retracted [28] (see Section 7.2 for tradeoffs).

2.2 Neural Network Model Merging

Model merging combines the parameters of two or more neural networks. Let θ_{base} denote base model parameters and $\tau_i = \theta_i - \theta_{\text{base}}$ the task vector for fine-tune i [12]. We evaluate 26 strategies spanning weight averaging [32], task arithmetic [12], TIES [33], DARE [37], Fisher merging [22], SLERP [30], evolutionary methods [1], and others (Appendix B).[1] Yang et al. [35] provide a comprehensive taxonomy. Federated learning [23, 15], the dominant framework for distributed model aggregation, relies on a central coordinator—creating a single point of failure [5]. A decentralised alternative requires the convergence guarantees that our CRDT wrapper provides.

3 The Problem: Why Direct CRDT on Tensors Fails

3.1 Formal Analysis of CRDT Property Violations

Let f denote a binary merge function on tensors. We require: $f(a, b) = f(b, a)$ (commutativity), $f(f(a, b), c) = f(a, f(b, c))$ (associativity), and $f(a, a) = a$ (idempotency). We present two representative strategy analyses here; the remaining strategies (TIES, DARE, Fisher, and others) are analysed in Appendix F.

Weight Averaging. Define $f(a, b) = (a + b)/2$ [32]. Commutativity and idempotency hold trivially. Associativity fails:

$$f(f(a, b), c) = \frac{a + b + 2c}{4} \quad (4)$$

$$f(a, f(b, c)) = \frac{2a + b + c}{4} \quad (5)$$

SLERP. For SLERP with parameter t [30], commutativity fails unless $t = 0.5$ (swapping inputs with fixed t changes the interpolation point). Associativity fails because composing geodesic interpolations changes the reference great circle. Idempotency holds: $\text{SLERP}(v, v; t) = v$ for all t.[2]

3.2 Incompatibility of Normalisation with CRDT Axioms

The failures above follow a structural pattern. We now show that normalisation—the operation at the heart of virtually every merge strategy—is incompatible with the full set of CRDT axioms.

Definition 3 (Normalising Merge Function)**.** *A binary merge function $f : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}^d$ is* normalising *if there exists a function $g : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}^d$ such that $f(a, b) = g(a, b)/n(a, b)$ where $n(a, b) \geq 1$ depends on the number or magnitude of the inputs. A merge function is* manifold-projecting *if its output is constrained to a*

[1] Of 26 strategies, 15 have peer-reviewed publications; 11 are derived/community strategies from MergeKit [10]. We include all to demonstrate universality.

[2] SLERP commutativity holds only at $t = 0.5$; the CRDT architecture resolves this for all t by canonically ordering inputs.

proper submanifold of $\mathbb{R}^d$ (e.g., the unit sphere). A merge function is thresholding *if it applies an input-dependent cutoff that discards components below a threshold computed from the inputs.*

Proposition 4 (Incompatibility of Normalisation with Associativity)**.** *Let $f : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}^d$ be a normalising merge function with $f(a, b) = g(a, b)/2$ for symmetric g. If g is not degenerate (i.e., $g(a, b)/2 \neq a$ for generic a, b), then f is not associative.*[3] *More generally, if f is manifold-projecting or thresholding, then f is not associative except in degenerate cases.*

Proof. We prove the result for count-based normalisation, then provide concrete counterexamples for projection and thresholding.

Suppose f normalises its output by dividing by the number of inputs being combined. For a pairwise merge, $f(a, b) = g(a, b)/2$ for some symmetric function g (to preserve commutativity). Consider associativity. When we compose pairwise merges, the left-association computes $f(f(a, b), c) = g\left(\frac{g(a,b)}{2}, c\right)/2$, which applies normalisation *twice*, each time with a divisor of 2, on different intermediate values. The right-association computes $f(a, f(b, c)) = g\left(a, \frac{g(b,c)}{2}\right)/2$. Because $g(a, b)/2 \neq a$ in general (unless g is degenerate), the intermediate values fed into the outer application of f differ between left- and right-association. Hence $f(f(a, b), c) \neq f(a, f(b, c))$ for generic a, b, c, violating associativity. (The weight averaging counterexample of Eqs. 4–5 provides the concrete instance.)

For manifold projection (e.g., SLERP at $t = 0.5$), we exhibit a concrete counterexample. Let $v_1 = (1, 0, 0)$, $v_2 = (0, 1, 0)$, $v_3 = (0, 0, 1)$ on S^2. Left-association computes $m_{12} = \text{SLERP}(v_1, v_2; 0.5) = \frac{1}{\sqrt{2}}(1, 1, 0)$, then $\text{SLERP}(m_{12}, v_3; 0.5)$, which lies on the great circle from $\frac{1}{\sqrt{2}}(1, 1, 0)$ to $(0, 0, 1)$. Right-association computes $m_{23} = \text{SLERP}(v_2, v_3; 0.5) = \frac{1}{\sqrt{2}}(0, 1, 1)$, then $\text{SLERP}(v_1, m_{23}; 0.5)$, which lies on a different great circle from $(1, 0, 0)$ to $\frac{1}{\sqrt{2}}(0, 1, 1)$. The two results are distinct unit vectors ($\approx (0.500, 0.500, 0.707)$ vs. $\approx (0.707, 0.500, 0.500)$, evaluated numerically), confirming that SLERP is not associative.

For thresholding (e.g., TIES trimming at 20%), let $a = (10, 1, 0.1)$, $b = (0.1, 10, 1)$, $c = (1, 0.1, 10)$ with a 20% trim (keeping the top 80% of magnitudes, i.e., dropping 1 of 3 components per vector). Left-association: trimming a and b yields $a' = (10, 1, 0)$ and $b' = (0, 10, 1)$; averaging gives $m_{ab} = (5, 5.5, 0.5)$. Then trimming m_{ab} and c yields $(5, 5.5, 0)$ and $(1, 0, 10)$; averaging gives the left result $\approx (3.0, 2.75, 5.0)$. Right-association: trimming b and c yields $(0, 10, 1)$ and $(1, 0, 10)$; averaging gives $m_{bc} = (0.5, 5.0, 5.5)$. Then trimming a and m_{bc} yields $(10, 1, 0)$ and $(0, 5, 5.5)$; averaging gives the right result $\approx (5.0, 3.0, 2.75)$. The two results differ ($\neq$), confirming non-associativity.

In all three cases, associativity is violated—confirming that normalisation, projection, and thresholding each independently break it. Since normalisation (in one of these forms) is a component of 25 of the 26 strategies we evaluated, this provides a structural explanation for the empirical observation that 0/26 strategies achieve system-level CRDT compliance on controlled 4×4 tensors (Table 3). □

We present Proposition 4 as a structural observation that explains the empirical finding of universal associativity failure (Table 3), rather than as a comprehensive impossibility theorem. A fully general characterisation of which merge functions can satisfy all three CRDT axioms simultaneously is an interesting open question.

3.3 Summary of Controlled Empirical Results

Controlled testing on 4×4 tensors (Table 3, Appendix A) confirms: 21/26 strategies are commutative, 14/26 idempotent, but only 1/26 (Task Arithmetic) is associative—and it fails idempotency. Zero out of 26 satisfy all three CRDT requirements. Associativity is the universal bottleneck (25/26 fail), structurally explained by Proposition 4. Section 6.2 confirms this pattern persists at production scale.

4 The Solution: Two-Layer Architecture

CRDT properties need not hold for tensor merge operations—only for the *state management layer* that determines which contributions are included. The actual merge strategy can be any deterministic function applied to a canonically-ordered set.

4.1 Architecture Overview

The CRDTMergeState architecture comprises two layers:

- **Layer 1 (CRDT State Management):** An OR-Set CRDT tracking model contributions. The merge operation is set union—trivially commutative, associative, and idempotent [29]. Version vectors provide causal ordering [19]; Merkle hash trees provide integrity verification [26].
- **Layer 2 (Deterministic Strategy Execution):** Given the converged contribution set, applies a merge strategy as a deterministic pure function. Canonical ordering (by content hash) and seeded randomness (from Merkle root) ensure identical results on all replicas.

[3] This covers weight averaging ($g(a, b) = a+b$, $f = (a+b)/2$); see Eqs. 4–5.

Layer 1 handles *what* to merge; Layer 2 handles *how*. Since Layer 2 is a pure function of Layer 1's converged state, the system converges. A worked data-flow example is provided in Appendix G.

4.2 Layer 1: CRDT State Management

Definition 5 (CRDTMergeState)**.** *A* CRDTMergeState *S is a tuple (A, R, V, H) where:*

- *A is the set of add entries—(e, t, n) triples where e is the model contribution, t is a unique tag, and n is the originating node;*
- *R is the set of remove entries: tags that have been removed;*
- *V is a version vector mapping node identifiers to logical timestamps [19];*
- *H is a Merkle hash tree over the visible elements [26].*

The visible set and merge operation are:

$$\text{Visible}(S) = \{e \mid \exists\,(e, t, n) \in A \text{ s.t. } t \notin R\} \tag{6}$$

$$\text{merge}(S_1, S_2) = (A_1 \cup A_2,\ R_1 \cup R_2,\ \max(V_1, V_2),\ H') \tag{7}$$

where $\max(V_1, V_2)$ is the component-wise maximum and H' is recomputed from the resulting visible set. In production, full-state merge should be replaced with delta-state propagation [2]; see Section 7.2.

Each contribution is identified by its SHA-256 hash [26], providing both deduplication and canonical ordering (a deterministic total order independent of insertion order or node identity). The Merkle tree over the visible set enables $O(\log n)$ convergence verification, efficient delta synchronisation, and provides a deterministic root hash for Layer 2's randomness requirements. Version vectors [19] track causal history, enabling detection of concurrent operations for OR-Set conflict resolution. Causal delivery is *not* required for correctness: the merge operation (Eq. 7) is commutative, associative, and idempotent, so messages may arrive in any order, be duplicated, or be delayed without affecting the converged state [29]. Version vectors serve an *optimisation* role—identifying which updates a peer already has to avoid redundant retransmission—not a correctness role.

4.3 Layer 2: Deterministic Strategy Execution

Definition 6 (Resolve Function)**.** *The resolve function $\mathcal{R} : 2^{\mathcal{M}} \times \Sigma \times \mathcal{H} \to \mathcal{M}$ takes a non-empty set of model contributions ($|C| \geq 1$), a strategy identifier $\sigma \in \Sigma$, and the Merkle root hash $h \in \mathcal{H}$, and returns a merged model:*

$$\mathcal{R}(C, \sigma, h) = \sigma(\text{sort}_{\text{hash}}(C),\ \text{seed}(h)) \tag{8}$$

Three mechanisms ensure determinism: (1) *canonical ordering* via SHA-256 content hashes defining a total order identical on all replicas; (2) *seeded randomness* derived from the Merkle root, ensuring identical seeds from identical contribution sets [26]; and (3) the *pure function guarantee* enforced by the API contract.

Remark 7 (N-way Generalisation)**.** *Strategies with natural n-ary forms (e.g., weight averaging, TIES) use them directly. Binary-only strategies (e.g., SLERP) are reduced via sequential fold over the canonical order:* $\text{fold}(\sigma, [c_1, \ldots, c_k])$*. The canonical ordering ensures this fold is identical on all replicas. Fold-based reduction introduces a weighting imbalance: for SLERP with $k = 3$ and $t = 0.5$, c_3 receives 50% weight versus 25% each for c_1, c_2 [30]. This does not affect CRDT compliance but may affect merge quality: for k contributions, the last element receives weight t while the first receives $t(1-t)^{k-2}$—exponential decay. Strategies with native n-ary forms avoid this issue. For binary-only strategies in large consortia, a balanced binary-tree reduction (depth $\lceil \log_2 k \rceil$) would equalise influence at the cost of a different—but still deterministic—reduction order.*

5 Mathematical Proof of CRDT Compliance

Let $\mathcal{S}$ denote the set of all CRDTMergeState instances. For $S \in \mathcal{S}$, let $\text{Visible}(S)$ denote the visible set (Eq. 6) and $\mathcal{R}(S) = \mathcal{R}(\text{Visible}(S), \sigma, h(S))$ the resolved value. Let $\sqcup : \mathcal{S} \times \mathcal{S} \to \mathcal{S}$ denote the merge operation (Eq. 7).

Theorem 8 (CRDT Compliance)**.** *The merge operation $\sqcup$ on* CRDTMergeState *satisfies commutativity, associativity, and idempotency. Moreover, $(\mathcal{S}, \sqsubseteq)$ is a join-semilattice with $\sqcup$ as the least upper bound, and* CRDTMergeState *is a CvRDT [29].*

Proof. Immediate from the OR-Set CvRDT result [29, 28]: our merge composes set union on A and R, component-wise max on V, and deterministic recomputation of H—all semilattice operations. The full verification is in Appendix C. □

We state three formal preconditions on merge strategies and the computational environment.

Assumption 9 (Strategy Purity)**.** *A merge strategy σ is a* pure function*: for all inputs (C, s), $\sigma(C, s)$ is uniquely determined by C and s, with no dependence on external state or non-deterministic operations beyond the provided seed s.*

Assumption 10 (Computational Determinism)**.** *All replicas execute σ using identical ISA, library versions, and IEEE 754 rounding mode (round-to-nearest-even), or use a fixed-precision format guaranteeing bitwise reproducibility.*

Assumption 11 (Collision Resistance)**.** *SHA-256 is collision-resistant: for distinct* $e_1 \neq e_2$, $\mathrm{SHA256}(e_1) \neq \mathrm{SHA256}(e_2)$ *with probability at least* $1 - 2^{-128}$.

Under these assumptions, we establish convergence through the following lemma and theorem (individual lemma proofs in Appendix D).

Lemma 12 (Determinism of Hashing, Ordering, and Seeding)**.** *If* $\mathrm{Visible}(S_1) = \mathrm{Visible}(S_2)$*, then under Assumption 11: (1) the hash sets are identical with distinct mappings; (2) the canonical orderings* $\mathrm{sort}_{\mathrm{hash}}$ *are equal; (3) the Merkle roots and derived seeds are equal.*

Theorem 13 (Convergence of Resolved Values)**.** *If* $\mathrm{Visible}(S_1) = \mathrm{Visible}(S_2)$ *and both use strategy* σ *under Assumptions 9–11, then* $\mathcal{R}(S_1) = \mathcal{R}(S_2)$.

Proof. By hypothesis, $\mathrm{Visible}(S_1) = \mathrm{Visible}(S_2)$. By Lemma 12, canonical orderings and seeds are equal. Since σ is a pure function (Assumption 9) receiving identical inputs (ordered contributions and seed) under identical computation (Assumption 10), outputs are identical:

$$\sigma\big(\mathrm{sort}_{\mathrm{hash}}(\mathrm{Visible}(S_i)),\ \mathrm{seed}(h(S_i))\big)$$

is the same for $i = 1, 2$. Therefore $\mathcal{R}(S_1) = \mathcal{R}(S_2)$. □

Corollary 14 (Universal CRDT-Compliant Merging)**.** *Every strategy* σ *satisfying Assumptions 9–11 can be used for CRDT-compliant merging through* CRDTMergeState*, regardless of its own algebraic properties.*

Proof. By Theorem 8, Layer 1 guarantees CRDT properties and convergence to identical visible sets. By Theorem 13, identical visible sets produce identical resolved values. The composed system satisfies Strong Eventual Consistency [29]. □

Theorem 15 (Complexity Bounds)**.** *For* k *contributions of* p *parameters each:* merge() *runs in* $O(|A_1| + |A_2|)$ *(independent of* p*);* add() *in* $O(p)$ *(SHA-256 hashing);* resolve() *in* $O(k \log k + T_\sigma(k, p))$ *where* T_σ *is the strategy cost. The CRDT overhead is* $O(k \log k)$ *time and* $O(k)$ *space, independent of model size* p.

6 Experimental Evaluation

We validate the formal specification in three tiers. *Tier 1* uses 4×4 tensors to verify algebraic properties in isolation. *Tier 2* scales to GPT-2-XL (1.5 B parameters) and Mistral-7B (7.24 B parameters) using independently published fine-tunes. *Tier 3* tests multi-node convergence under gossip protocols and network partitions. Since the CRDT guarantees (Theorems 8–13) are algebraic properties independent of tensor dimensions, Tier 1 suffices for correctness; Tier 2 confirms the implementation generalises and quantifies overhead.

6.1 Tier 1: Controlled Algebraic Verification

We evaluated all 26 strategies on 4×4 float64 tensors (seed 42, tolerance 10^{-5}) under both raw operations (Phase 1) and the CRDTMergeState architecture (Phase 2). Phase 1 (Table 3, Appendix A) confirms the theoretical analysis: 21/26 strategies are commutative, 14/26 idempotent, only 1/26 (Task Arithmetic) is associative—and it fails idempotency. System-level CRDT compliance: 0/26. Phase 2 yields 26/26 strategies passing all four CRDT properties (commutativity, associativity, idempotency, 3-replica convergence)—104/104 individual tests.

6.2 Tier 2: Production-Scale Validation

6.2.1 Models and Data

We tested on two transformer language models with independently published fine-tunes providing genuine weight divergence:

- **GPT-2-XL** (1.5 B params, 193 eligible 2D layers) [25] with three independently published fine-tunes (*instruct*, *domain*, *wiki*).[4]
- **Mistral-7B-v0.1** (7.24 B params, 224 eligible 2D layers) [13] with three fine-tunes (*instruct*, *hermes*, *zephyr*; Appendix H).

All weights stored in float16, cast to float64 for testing to isolate algebraic violations from rounding artefacts. Experiments ran on a single NVIDIA A100-SXM4-80GB with PyTorch 2.10.0 and CUDA 12.8.

6.2.2 Testing Methodology

For each strategy and model, CRDT properties are tested via slice-based evaluation: a representative 128×128 slice per unique tensor shape, with results extrapolated to all layers sharing that shape. This approach tests the strategy's algebraic behaviour on realistic weight distributions from production models, though it does not exercise the full dimensionality of each layer. Capped verification at 512×512 confirms consistency at higher resolution; mismatches are flagged (see `ada_merging` discussion below, which demonstrates that this cross-resolution check catches real violations). Tolerance is $\mathrm{atol} = 10^{-5}$. Phase 2 additionally tests 3-replica convergence over all six merge-order permutations.

6.2.3 Phase 1 Results: Raw Strategy Properties at Scale

The core finding is unchanged from controlled experiments: *associativity remains the dominant failure mode*, with 22–25 of 26 strategies failing associativity across

[4] Full HuggingFace model identifiers are listed in Appendix H.

Table 1: Tier 2, Phase 1 Results: Raw CRDT property compliance at production scale. P = Pass (all tested layers pass), F = Fail. † SLERP commutativity tested at $t = 0.5$. ∗ Verification mismatch between 128×128 slice and 512×512 capped test (see Section 6.3).

	GPT-2-XL (1.5 B)				Mistral-7B (7.24 B)			
Strategy	C	A	I	CRDT?	C	A	I	CRDT?
ada merging	P	P*	P	P*	P	P	P	P
adarank	P	F	F	F	P	F	F	F
dam	P	F	P	F	P	F	P	F
dare	F	F	F	F	F	F	F	F
dare ties	F	F	F	F	F	F	F	F
della	F	F	F	F	F	F	F	F
dual projection	P	F	P	F	P	F	P	F
emr	P	F	F	F	P	F	F	F
evolutionary merge	F	F	F	F	F	F	F	F
fisher merge	P	F	P	F	P	F	P	F
genetic merge	P	F	P	F	P	P	P	P
led merge	P	P	P	P	P	P	P	P
linear	P	F	P	F	P	F	P	F
model breadcrumbs	P	F	F	F	P	F	F	F
negative merge	P	F	F	F	P	F	F	F
regression mean	P	F	P	F	P	F	P	F
repr. surgery	P	F	P	F	P	F	P	F
safe merge	P	F	P	F	P	F	P	F
slerp†	P	F	P	F	P	F	P	F
split unlearn merge	P	F	F	F	P	F	F	F
star	P	F	F	F	P	F	F	F
svd knot tying	F	F	P	F	F	F	P	F
task arithmetic	P	P	F	F	P	P	F	F
ties	P	F	F	F	P	F	F	F
weight average	P	F	P	F	P	F	P	F
weight scope align.	P	F	P	F	P	F	P	F
Totals	21	3	14	2	21	4	14	3

both models and scales. The 2–3 strategies that newly pass associativity at production scale (and the 2–3 that achieve all three properties simultaneously) represent numerical coincidence on specific weight distributions, not algebraic compliance (Section 6.3).

6.2.4 Phase 2 Results: CRDTMergeState at Scale

All 26 strategies achieve 100% CRDT compliance through the two-layer architecture on both models: 104 strategy-level tests per model (26 strategies × 4 properties), verified across 193 layers (GPT-2-XL) and 224 layers (Mistral-7B). In total, 43,368 layer-level property checks pass at capped tensor resolution (128×128 slices, with 512×512 capped verification). Full-layer verification on a representative subset—6 strategies (weight averaging, task arithmetic, TIES, DARE, SLERP, Fisher merging) covering all strategy categories (linear, stochastic, binary-fold) across the 10 largest weight matrices per model (up to 6144×1600 for Mistral-7B)—confirms consistency with the slice-based results (see the `ada_merging` analysis in Section 6.3). Combined with Tier 1 (104 controlled tests), the architecture achieves 100% compliance across 312 strategy-level tests and $43{,}368 + 104 = 43{,}472$ total layer-level evaluations.

6.3 Cross-Scale Analysis

Table 2 summarises CRDT property compliance across all three evaluation scales.

Commutativity (21/26) and idempotency (14/26) are stable across all three scales, confirming these properties are determined by algorithmic structure. Associativity varies at the margin: 2–3 strategies that fail on controlled 4×4 tensors pass associativity within floating-point tolerance at production scale (1/26→3/26 on GPT-2-XL; 1/26→4/26 on Mistral-7B). The `ada_merging` verification mismatch (marked ∗ in Table 1)—where associativity passes within tolerance on 128×128 slices but fails on 512×512 slices of the same GPT-2-XL weight matrix—demonstrates that empirical compliance is resolution-dependent: the associativity violation is real but small enough to fall within $\text{atol} = 10^{-5}$ at low resolution, only surfacing at higher resolution where the accumulated error exceeds tolerance. This fragility is why *algebraic guarantees* (Corollary 14), not empirical coincidence, are necessary for distributed systems requiring

Table 2: Cross-scale summary of raw Phase 1 CRDT property compliance (consolidating Tables 3 and 1). Commutativity and idempotency rates are stable across scales; associativity shows minor variation (see text). ∗ Strategies passing all 3 properties at production scale represent *numerical coincidence* on specific weight distributions—not algebraic compliance—as demonstrated by the `ada_merging` verification mismatch (Section 6.3). The two-layer architecture provides the only *guaranteed* compliance (26/26 at all scales).

Scale	Layers	C	A	I	All 3
Controlled (4×4)	—	21/26	1/26	14/26	0/26
GPT-2-XL (1.5 B)	193	21/26	3/26	14/26	2/26*
Mistral-7B (7.24 B)	224	21/26	4/26	14/26	3/26*
CRDTMergeState (Phase 2): 26/26 at all three scales.					

hard convergence. The phenomenon itself—associativity violations that vanish in high-dimensional spaces—merits further investigation. One hypothesis is that weight distributions in large models concentrate near low-rank manifolds where the nonlinear components of merge operations (normalisation, projection) become approximately linear, reducing the left–right association gap below floating-point tolerance. A formal characterisation of when "approximate associativity" emerges could inform the design of strategies that are inherently closer to CRDT-compliant.

6.4 Performance Overhead

The CRDT layer introduces negligible overhead: merge() is sub-millisecond regardless of model size (set operations only); add() is dominated by SHA-256 hashing ($O(p)$); resolve() CRDT overhead (sorting, Merkle root, seed derivation) is consistently below 0.5 ms, with total latency dominated by the strategy itself. Memory overhead is below 10 KB for 16 contributions. Scalability benchmarks on the A100 confirm linear scaling in parameter count, consistent with $O(k \log k + T_\sigma(k, p))$. This overhead is dwarfed by inference, fine-tuning, and strategy execution costs.

6.5 Tier 3: Multi-Node Convergence Suite

To validate convergence under realistic distributed conditions, we execute a four-part convergence suite using the `crdt-merge` library (v0.9.4). The gossip protocol is *push-based all-pairs*: in each round, every node sends its full CRDT state to every other node, which merges it locally via Eq. 7. For n nodes this requires $n(n-1)$ directed merge calls per round; since each call is $O(|A_1|+|A_2|)$ (set union, independent of tensor size), the gossip phase scales as $O(n^2)$ in node count while remaining $O(1)$ in model size. This all-pairs protocol is a *prototype for validation purposes*, chosen as the simplest correct implementation; production deployments should use epidemic (randomised) gossip [18], which reduces per-round communication to $O(n)$ at the cost of slower convergence and is a natural scalability optimisation beyond ∼50 nodes. Full results appear in Appendix I.

Multi-node convergence. One hundred nodes each contribute a 512×512 tensor (26,214,400 parameters per contribution) using `slerp`. Across 20 random gossip orderings, all nodes converge to a bitwise-identical result (max element-wise difference = 0), with average gossip time 492.8 ms and average resolve time ∼19.7 s (Table 6).

Partition healing. The 100 nodes are split into 10 isolated partitions (10 nodes each). Each partition converges internally to a distinct hash. After healing, full gossip resumes and all 100 nodes converge to a single bitwise-identical result, confirming Strong Eventual Consistency under network partitions.

Cross-strategy sweep. All 26 strategies are tested on 10 nodes with 64×64 tensors. For each strategy, all 10 nodes converge to the same final hash (Table 8), confirming that convergence is strategy-independent.

Scalability. Convergence is verified from 2 to 50 nodes. Gossip time scales as $O(n^2)$ (expected for all-pairs merge), while per-call `merge()` cost remains $O(1)$ in tensor size. All scales achieve 100% convergence (Table 9). At k=200 contributions, gossip requires $200 \times 199 = 39{,}800$ merge calls (directed pairs, consistent with the all-pairs protocol above); since each call is $O(1)$ in tensor size (set union only), the gossip phase remains tractable even for large consortia.

7 Discussion

Remark 16 (Downstream Equivalence)**.** *The CRDT wrapper does not modify the merged model. Layer 2 applies the identical merge strategy σ to the identical ordered contribution set with the identical random seed (Theorem 13). Therefore, the downstream task performance of CRDT-wrapped merging is identical by construction to that of non-CRDT merging using the same strategy, contributions, and ordering. Evaluating downstream performance of specific strategies is a question about the strategies themselves—not the*

CRDT architecture, which is transparent to the merge computation. We verified this empirically: for representative strategy–model pairs, the CRDT-wrapped resolve() *produces byte-identical output to a direct (unwrapped) strategy call on the same canonically-ordered inputs and seed, confirming that the wrapper introduces zero computational divergence. The extensive literature on merge strategy quality [35, 32, 12, 33, 37, 22, 1, 10] applies directly to CRDT-wrapped merges. A single downstream benchmark (e.g., MMLU or HellaSwag) would provide additional empirical confirmation of this transparency property; we defer such evaluation to follow-up work focused on strategy selection, as it is orthogonal to the convergence guarantees established here.*

Associativity as the Dominant Failure Point. Associativity is the fundamental obstacle to CRDT compliance: 22–25 of 26 strategies fail across all scales (Table 2). Proposition 4 explains this structurally: normalisation inherently breaks associativity. The cross-scale analysis (Section 6.3) further shows that the few strategies passing empirically at production scale represent numerical coincidence—resolution-dependent and fragile—underscoring the need for algebraic guarantees over empirical testing.

Implications for Federated and Decentralised Learning. The architecture enables fully asynchronous, peer-to-peer model merging with guaranteed convergence, complementing—but not replacing—federated learning [23]. While FL guarantees convergence of *training* under data distribution assumptions with a central coordinator [15, 20], CRDT-Merge guarantees convergence of *state* under computational determinism with no coordinator. Neither subsumes the other: the approaches are complementary. Decentralised FL via gossip protocols [21, 18] could use CRDT-Merge for aggregation. The domain-specific engineering—Merkle-root-derived seeding, SHA-256-based canonical ordering, and explicit handling of binary-to-n-ary reduction—represents the domain-specific contribution beyond the known CRDT composition pattern [28].

7.1 Floating-Point Determinism

Theorem 13 requires Assumption 10: all replicas must produce bitwise-identical results. In practice, this is satisfied by containerised deployment with identical binaries and hardware, deterministic CUDA operations, or quantised representations [27]. Our experiments ran on a single GPU type (A100-SXM4-80GB) and therefore do not assess cross-hardware reproducibility; cross-architecture validation (e.g., A100 vs. H100 vs. CPU) is a necessary next step before heterogeneous deployment.

We propose a concrete fallback protocol: after each resolve(), every replica broadcasts its Merkle root of the resolved output (a single 256-bit hash). If all roots agree, convergence is confirmed—our Tier 3 experiments (100 nodes, 20 orderings) demonstrate that this is the common case under homogeneous deployment. If roots disagree—indicating Assumption 10 is violated—replicas fall back to the output of a designated reference replica (lowest node ID), reducing the guarantee from independent convergence to agreement on a reference computation while preserving SEC at the cost of one additional round. When active, this fallback degrades the system from fully decentralised SEC to coordinator-assisted SEC for the resolve() step only; the state management layer (Layer 1) remains fully decentralised and coordinator-free. We state this scope explicitly: the "coordinator-free" claim applies unconditionally to state convergence (Layer 1) but conditionally to resolved-value agreement (Layer 2), contingent on Assumption 10.

7.2 Limitations and Future Work

We group limitations into four categories (expanded in Appendix E).

L1: Deployment Constraints. Correctness requires strategy purity (Assumption 9) and computational determinism (Assumption 10), enforced through seeded randomness, canonical ordering, and containerised deployment. For billion-parameter models, delta-state CRDTs [2] are essential for practical deployment (not yet implemented in the current prototype; adaptation is straightforward—see Appendix E); version vectors scale as $O(n)$ in nodes, replaceable by dotted version vectors [24] for $n > 1{,}000$.

L2: Semantic Evaluation. The architecture guarantees *syntactic* convergence but does not evaluate downstream task performance. As Remark 16 establishes, the CRDT wrapper is transparent, so strategy quality literature applies directly.

L3: Scalability. The system recomputes the merged model from the full contribution set on every resolve() call ($O(k \cdot p)$ cost); incremental strategies are needed for very large contribution sets. Three mitigation paths exist: (1) caching the resolved output and invalidating only when the contribution set changes; (2) hierarchical resolve, where sub-groups resolve locally and a second pass merges sub-group outputs; and (3) strategies with algebraic structure permitting incremental updates (e.g., weight averaging admits $O(p)$ updates per new contribution). Tombstone garbage collection via causal stability analysis [3] prevents unbounded metadata growth; GC must be deferred until after resolve() has been executed and its output disseminated, ensuring all replicas resolve against the same visible set before metadata is pruned. We have not empirically evaluated tombstone accumulation rates; for the consortium scenario ($k < 100$

contributions, infrequent removals), tombstone overhead is negligible, but long-running deployments with frequent model retraction would benefit from empirical GC characterisation.

L4: Security and Conflict Resolution. The OR-Set's add-wins policy means concurrent adds survive concurrent removes—problematic if removal represents discovery of a poisoned model. The architecture does not currently provide Byzantine fault tolerance. However, the two-layer separation creates a structural opportunity that goes beyond incremental hardening: trust metadata—such as evidence of equivocation (a node sending inconsistent states to different peers), Merkle-root divergence, or contribution-fingerprint anomalies—can itself be modelled as a monotonically-growing CRDT within Layer 1. A gated merge at the Layer 2 boundary would then reject contributions from nodes whose converged trust score falls below a configurable threshold.

Critically, trust convergence would be guaranteed by the same join-semilattice proof as data convergence, enabling consensus-free Byzantine isolation at well-defined threat levels: given n nodes with at most f Byzantine actors, if evidence propagation reaches all honest nodes (guaranteed under eventual delivery), then all $n - f$ honest nodes converge to the same trust state, make identical gating decisions, and therefore converge to identical data states—without coordinators, external oracles, or cryptographic voting rounds. This would yield a promising structural pathway toward Byzantine exclusion at well-defined threat levels: rather than degrading gracefully, the system could provably exclude Byzantine contributions from the merged output once the evidence lattice has converged. The exclusion threshold, evidence types, and convergence bounds would be configurable parameters defining the exclusion level for a given deployment.

This "self-securing lattice" pattern—where the system uses its own convergence guarantees to secure itself—is structurally enabled by the two-layer architecture and impossible in single-layer CRDT designs where trust and data are entangled. Preliminary analysis suggests this is structurally feasible; integration with existing Byzantine-resilient aggregation methods [4] is also possible as a complementary defence layer.

8 Related Work

Model Merging. Yang et al. [35] provide a comprehensive survey; MergeKit [10] is the standard toolkit. Git-Theta [16] provides version control for model parameters via Git but requires a central server, assumes a single canonical branch, and provides no conflict-free merge semantics—when two participants independently merge the same models, Git-Theta has no mechanism to guarantee convergent results. Our architecture provides exactly this guarantee: any number of replicas can independently merge in any order and provably converge to identical states. No prior work addresses the algebraic CRDT properties required for conflict-free distributed model merging.

Distributed Systems and CRDTs. CRDTs [29, 28] have been extensively studied [24, 17], with Merkle-CRDTs [26] combining Merkle-DAGs with CRDT semantics. The pattern of composing CRDTs with deterministic functions is known [28]. Our contribution is (a) identifying that model merging requires this pattern due to universal associativity failure, (b) the specific construction combining OR-Set semantics with content-addressable hashing, Merkle trees, and seeded randomness for neural network parameters, and (c) the systematic algebraic audit of 26 strategies motivating the architecture. Delta-state CRDTs [2] offer efficient synchronisation; adapting our architecture to delta-state propagation is a natural deployment optimisation.

Federated Learning. All centralised FL systems [23, 15, 20, 5] assume a central coordinator. Decentralised FL via gossip protocols [21, 18] focuses on training convergence, not the state convergence guarantees we establish.

Patents. The two-layer CRDT architecture is the subject of UK Patent Application No. 2607132.4 [9].[5] No prior patent or publication combines CRDT theory with neural network model merging.

9 Conclusion

We have shown that neural network model merging universally fails the algebraic requirements for conflict-free distributed operation—a structural impossibility for normalisation-based strategies, not merely an implementation limitation—and presented a two-layer architecture that resolves this by separating CRDT state management from deterministic strategy execution, achieving compliance across all 26 strategies tested. Our formal analysis established that no strategy satisfies the required commutativity, associativity, and idempotency, with Proposition 4 providing a structural explanation through normalisation incompatibility. Our two-layer architecture—CRDTMergeState—resolves this by separating CRDT state management (Layer 1, OR-Set with version vectors and Merkle hashing) from deterministic strategy execution (Layer 2), with formal proofs of CvRDT compliance (Theorem 8), convergence (Theorem 13), and bounded overhead (Theorem 15). Empirical validation confirmed 100% compliance across

[5] Disclosed for transparency and to establish prior art; the theoretical contributions of this paper are independent of the patent application.

two tiers (312 strategy-level tests, 43,472 layer-level evaluations) with overhead below 0.5 ms. This enables decentralised, coordinator-free model merging with formal convergence guarantees, advancing peer-to-peer model development [29, 35]. Future directions include Byzantine exclusion via trust-as-CRDT mechanisms—where trust evidence would converge through the same lattice guarantees as data, offering a promising pathway toward provable Byzantine exclusion at known threat levels without consensus protocols (Section 7.2, L4)—as well as delta-state optimisation, cross-hardware determinism validation, incremental resolve strategies for large contribution sets, and integration with federated learning pipelines.

Ethics Statement

This work introduces infrastructure for decentralised model merging and does not involve human subjects, private data, or dual-use capabilities. We identify no direct negative societal impacts from the CRDT architecture itself; however, decentralised merging could facilitate uncontrolled model combination without quality assurance. Section 7.2 (L4) discusses adversarial considerations and mitigation strategies.

Reproducibility Statement

The `crdt-merge` library (v0.9.4) and accompanying verification notebook are available at `https://github.com/RyanGillespie/crdt-merge`. All experiments use publicly available models from HuggingFace (Appendix H) and a single NVIDIA A100-SXM4-80GB GPU. The complete test suite, including the Tier 1–3 verification scripts, will be open-sourced upon publication.

Acknowledgments

The author thanks the anonymous reviewers for their detailed feedback across three revision cycles.

References

[1] Takuya Akiba, Makoto Shing, Yujin Tang, Qi Sun, and David Ha. Evolutionary optimization of model merging recipes. *Nature Machine Intelligence*, 2025.

[2] Paulo Sérgio Almeida, Ali Shoker, and Carlos Baquero. Delta state replicated data types. *Journal of Parallel and Distributed Computing*, 111, 2018.

[3] Carlos Baquero, Paulo Sérgio Almeida, and Ali Shoker. Making operation-based CRDTs operation-based. In *Proc. DAIS*, 2014.

[4] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Proc. NeurIPS*, 2017.

[5] Keith Bonawitz et al. Towards federated learning at scale: A system design. In *Proc. MLSys*, 2019.

[6] MohammadReza Davari and Eugene Belilovsky. Model breadcrumbs: Scaling multi-task model merging with sparse masks. In *Proc. ECCV*, 2024.

[7] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, et al. Dynamo: Amazon's highly available key-value store. In *Proc. SOSP*, 2007.

[8] Pala Tej Deep, Rishabh Bhardwaj, and Soujanya Poria. DELLA-merging: Reducing interference in model merging through magnitude-based sampling. *arXiv:2406.11617*, 2024.

[9] Ryan Gillespie. Method and system for conflict-free merging of neural network model parameters using convergent replicated data types. UK Patent Application No. 2607132.4, 2026. Filed 30 March 2026.

[10] Charles Goddard, Shamane Siriwardhana, Malikeh Ehghaghi, Luke Meyers, et al. Arcee's MergeKit: A toolkit for merging large language models. In *Proc. EMNLP Industry Track*, 2024.

[11] Chenyu Huang, Peng Ye, Tao Chen, Tong He, Xiangyu Yue, and Wanli Ouyang. EMR-merging: Tuning-free high-performance model merging. In *Proc. NeurIPS*, 2024.

[12] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *Proc. ICLR*, 2023.

[13] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, et al. Mistral 7B. *arXiv:2310.06825*, 2023.

[14] Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless knowledge fusion by merging weights of language models. In *Proc. ICLR*, 2023.

[15] Peter Kairouz, H. Brendan McMahan, et al. Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, 14(1–2), 2021.

[16] Nikhil Kandpal, Brian Lester, Mohammed Muqeeth, Anisha Mascarenhas, Monty Evans, Vishal Baskaran, Tenghao Huang, Haokun Liu, and Colin Raffel. Gittheta: A git extension for collaborative development of machine learning models. In *Proc. ICML*, 2023.

[17] Martin Kleppmann and Alastair R. Beresford. A conflict-free replicated JSON datatype. *IEEE Trans. Parallel and Distributed Systems*, 28(10), 2017.

[18] Anastasia Koloskova, Sebastian U. Stich, and Martin Jaggi. Decentralized stochastic optimization and gossip algorithms with compressed communication. In *Proc. ICML*, 2019.

[19] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 1978.

[20] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proc. MLSys*, 2020.

[21] Xiangru Lian, Ce Zhang, Huan Zhang, Cho-Jui Hsieh, Wei Zhang, and Ji Liu. Can decentralized algorithms outperform centralized algorithms? A case study for decentralized parallel stochastic gradient descent. In *Proc. NeurIPS*, 2017.

[22] Michael S. Matena and Colin Raffel. Merging models with Fisher-weighted averaging. In *Proc. NeurIPS*, 2022.

[23] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proc. AISTATS*, 2017.

[24] Nuno Preguiça, Carlos Baquero, and Marc Shapiro. Conflict-free replicated data types (CRDTs). In *Encyclopedia of Big Data Technologies*. Springer, 2018.

[25] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.

[26] Hector Sanjuan, Samuli Poyhtari, Pedro Teixeira, and Ioannis Psaras. Merkle-CRDTs: Merkle-DAGs meet CRDTs. *arXiv:2004.00107*, 2020.

[27] Fred B. Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys*, 22(4), 1990.

[28] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. A comprehensive study of convergent and commutative replicated data types. Technical Report RR-7506, INRIA, 2011.

[29] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Proc. SSS*, 2011.

[30] Ken Shoemake. Animating rotation with quaternion curves. In *Proc. SIGGRAPH*, 1985.

[31] Werner Vogels. Eventually consistent. *Communications of the ACM*, 52(1), 2009.

[32] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *Proc. ICML*, 2022.

[33] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-merging: Resolving interference when merging models. In *Proc. NeurIPS*, 2023.

[34] Enneng Yang et al. Representation surgery for multi-task model merging. In *Proc. ICML*, 2024.

[35] Enneng Yang, Li Shen, Zhenyi Wang, Guibing Guo, Xingwei Wang, Xiaochun Cao, Jie Zhang, and Dacheng Tao. Model merging in LLMs, MLLMs, and beyond: Methods, theories, applications and opportunities. *ACM Computing Surveys*, 2026.

[36] Enneng Yang, Zhenyi Wang, Li Shen, et al. AdaMerging: Adaptive model merging for multi-task learning. In *Proc. ICLR*, 2024.

[37] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super Mario: Absorbing abilities from homologous models as a free lunch. In *Proc. ICML*, 2024.

A Controlled Verification Results

This appendix presents the full per-strategy results for Tier 1 (controlled 4×4 tensor) evaluation. Table 3 shows Phase 1 (raw tensor operations) and Table 4 shows Phase 2 (two-layer architecture). These results are summarised in the main text (Section 6.1) and in the cross-scale comparison (Table 2).

B Strategy Descriptions and Provenance

Of the 26 strategies evaluated, 15 have direct peer-reviewed publications: weight averaging [32], task arithmetic [12], TIES [33], DARE [37], DARE-TIES, Fisher merging [22], SLERP [30], AdaMerging [36], DELLA [8], RegMean [14], EMR-Merging [11], Model Breadcrumbs [6], Representation Surgery [34], evolutionary merge [1], and linear interpolation. The remaining 11 are either derived strategies—combinations or variants of established methods (ADArank, DAM, dual projection, genetic merge, LED merge, negative merge, safe merge, split–unlearn merge)—or community toolkit utilities from MergeKit [10] (STAR, SVD knot tying, weight scope alignment).

Key equations for formal analysis:

- **Weight Averaging / Model Soups:** $\theta_{\text{merged}} = \frac{1}{n}\sum_{i=1}^{n}\theta_i$ [32].
- **Task Arithmetic:** $\theta_{\text{merged}} = \theta_{\text{base}} + \lambda \sum_{i=1}^{n}\tau_i$ where $\tau_i = \theta_i - \theta_{\text{base}}$ [12].
- **TIES-Merging:** Three-step pipeline: (1) trim low-magnitude values, (2) resolve sign conflicts via majority vote, (3) merge agreed-upon components [33].
- **DARE:** Random dropout with probability p and rescaling by $1/(1-p)$ [37].
- **Fisher-Weighted Merging:** $\theta_{\text{merged}} = \frac{\sum_{i=1}^{n} F_i \odot \theta_i}{\sum_{i=1}^{n} F_i}$ where F_i is the diagonal Fisher information [22].
- **SLERP [30]:** Spherical linear interpolation between unit vectors: $\text{SLERP}(v_1, v_2; t)$ interpolates along the great circle with angle $\Omega = \arccos(\hat{v}_1 \cdot \hat{v}_2)$.
- **Evolutionary Merging:** Population-based search over per-layer merge coefficients [1].
- **Additional:** AdaMerging [36] (adaptive coefficients), DELLA [8] (magnitude-based sampling), RegMean [14] (regression-based mean), EMR-Merging [11] (elect-mask-rescale), Model Breadcrumbs [6] (sparse masks), Representation Surgery [34] (representation bias resolution).

C Proof of CvRDT Compliance (Theorem 8)

We verify each CRDT property by reduction to set union and component-wise maximum, both well-known semilattice operations [28].

We define a partial order $\sqsubseteq$ on $\mathcal{S}$ by

$$S_1 \sqsubseteq S_2 \iff A_1 \subseteq A_2 \ \wedge\ R_1 \subseteq R_2 \ \wedge\ V_1 \leq V_2, \quad (9)$$

where $V_1 \leq V_2$ denotes component-wise $\leq$ on version vectors.

Remark 17 (Partial Order and Visible Sets)**.** *The partial order $\sqsubseteq$ is defined on the metadata (A, R, V) rather than on visible sets directly, consistent with the standard OR-Set formulation [29, 28]. Monotonic growth of A and R under $\sqsubseteq$ ensures information content increases monotonically, even though* Visible(S) *may shrink as removes are incorporated—receiving a remove is new information, and the partial order correctly reflects this.*

Commutativity. For states S_1, S_2:

$$S_1 \sqcup S_2 = (A_1 \cup A_2,\ R_1 \cup R_2,\ \max(V_1, V_2),\ H') \quad (10)$$
$$S_2 \sqcup S_1 = (A_2 \cup A_1,\ R_2 \cup R_1,\ \max(V_2, V_1),\ H'') \quad (11)$$

Since set union is commutative ($A_1 \cup A_2 = A_2 \cup A_1$) and component-wise max is commutative ($\max(V_1, V_2) = \max(V_2, V_1)$), we have $S_1 \sqcup S_2 = S_2 \sqcup S_1$ (with $H' = H''$ as both are deterministic functions of the same visible set).

Associativity. For states S_1, S_2, S_3:

$$(S_1 \sqcup S_2) \sqcup S_3 = (A_1 \cup A_2 \cup A_3, \ldots) \quad (12)$$
$$S_1 \sqcup (S_2 \sqcup S_3) = (A_1 \cup A_2 \cup A_3, \ldots) \quad (13)$$

By associativity of set union and component-wise max, all components are equal.

Idempotency. For state S:

$$S \sqcup S = (A \cup A,\ R \cup R,\ \max(V, V),\ H') = (A, R, V, H) = S \quad (14)$$

by idempotency of set union and max.

Least Upper Bound. Under the partial order $\sqsubseteq$ (Eq. 9), $S_1 \sqcup S_2$ satisfies $S_1 \sqsubseteq S_1 \sqcup S_2$ and $S_2 \sqsubseteq S_1 \sqcup S_2$ (since $A_i \subseteq A_1 \cup A_2$, etc.). For any upper bound S' with $S_1 \sqsubseteq S'$ and $S_2 \sqsubseteq S'$, we have $A_1 \cup A_2 \subseteq A'$, $R_1 \cup R_2 \subseteq R'$, and $\max(V_1, V_2) \leq V'$, so $S_1 \sqcup S_2 \sqsubseteq S'$. Hence $S_1 \sqcup S_2$ is the least upper bound, confirming the semilattice structure. $\square$

Table 3: Tier 1, Phase 1 Results: CRDT property compliance of raw merge operations on 4×4 tensors. P = Pass, F = Fail. No strategy achieves system-level CRDT compliance (all three properties simultaneously). †SLERP commutativity tested at $t = 0.5$; fails for $t \neq 0.5$.

Strategy	Commut.	Assoc.	Idemp.	CRDT?
ada merging	P	F	P	F
adarank	P	F	F	F
dam	P	F	P	F
dare	F	F	F	F
dare ties	F	F	F	F
della	F	F	F	F
dual projection	P	F	P	F
emr	P	F	F	F
evolutionary merge	F	F	F	F
fisher merge	P	F	P	F
genetic merge	P	F	P	F
led merge	P	F	P	F
linear	P	F	P	F
model breadcrumbs	P	F	F	F
negative merge	P	F	F	F
regression mean	P	F	P	F
repr. surgery	P	F	P	F
safe merge	P	F	P	F
slerp†	P	F	P	F
split unlearn merge	P	F	F	F
star	P	F	F	F
svd knot tying	F	F	P	F
task arithmetic	P	P	F	F
ties	P	F	F	F
weight average	P	F	P	F
weight scope alignment	P	F	P	F
Totals	21/26	1/26	14/26	0/26

D Individual Determinism Lemmas

Lemma 12 in the main text consolidates three properties. We state and prove each individually here.

Hash Determinism. SHA-256 is a deterministic function. For any model contribution e, the hash $\text{SHA256}(e)$ is uniquely determined by e. Consequently, if $\text{Visible}(S_1) = \text{Visible}(S_2)$, then $\{\text{SHA256}(e) : e \in \text{Visible}(S_1)\} = \{\text{SHA256}(e) : e \in \text{Visible}(S_2)\}$. By Assumption 11, distinct contributions map to distinct hashes with overwhelming probability.

Proof. SHA-256 is a standardised cryptographic hash function (NIST FIPS 180-4): a deterministic, stateless mapping from arbitrary-length byte strings to 256-bit digests. The conclusion follows from $\text{Visible}(S_1) = \text{Visible}(S_2)$ and Assumption 11. □

Ordering Determinism. Let $\text{sort}_{\text{hash}}$ denote sorting by SHA-256 content hash. If $\text{Visible}(S_1) = \text{Visible}(S_2)$ then

$$\text{sort}_{\text{hash}}(\text{Visible}(S_1)) = \text{sort}_{\text{hash}}(\text{Visible}(S_2)).$$

By Assumption 11, the ordering is a total order on contributions (no ties) with overwhelming probability.

Proof. Sorting is deterministic on totally ordered sets. By hash determinism, the hash values (and hence the total order) are identical. Therefore the sorted sequences are equal. □

Seed Determinism. The Merkle hash tree is computed deterministically from the canonically-ordered elements [26]. If the canonical orderings are equal, then: (1) the Merkle roots are equal: $h(S_1) = h(S_2)$; and (2) the derived seeds are equal: $\text{seed}(h(S_1)) = \text{seed}(h(S_2))$.

Proof. The Merkle tree is constructed by recursively hashing pairs of child nodes. Since the leaf nodes (the canonically-ordered contribution hashes) are identical by ordering determinism, all intermediate and root hashes are identical. The seed derivation function is a deterministic transformation of the root hash. □

E Detailed Limitations

The main text (Section 7.2) groups limitations into four categories. We expand each here.

Table 4: Tier 1, Phase 2 Results: CRDT property compliance through the CRDTMergeState two-layer architecture on 4×4 tensors. P = Pass. All 26 strategies pass all 4 properties (104/104 tests).

Strategy	Commut.	Assoc.	Idemp.	Conv.	CRDT?
ada merging	P	P	P	P	✓
adarank	P	P	P	P	✓
dam	P	P	P	P	✓
dare	P	P	P	P	✓
dare ties	P	P	P	P	✓
della	P	P	P	P	✓
dual projection	P	P	P	P	✓
emr	P	P	P	P	✓
evolutionary merge	P	P	P	P	✓
fisher merge	P	P	P	P	✓
genetic merge	P	P	P	P	✓
led merge	P	P	P	P	✓
linear	P	P	P	P	✓
model breadcrumbs	P	P	P	P	✓
negative merge	P	P	P	P	✓
regression mean	P	P	P	P	✓
repr. surgery	P	P	P	P	✓
safe merge	P	P	P	P	✓
slerp	P	P	P	P	✓
split unlearn merge	P	P	P	P	✓
star	P	P	P	P	✓
svd knot tying	P	P	P	P	✓
task arithmetic	P	P	P	P	✓
ties	P	P	P	P	✓
weight average	P	P	P	P	✓
weight scope alignment	P	P	P	P	✓
Totals	26/26	26/26	26/26	26/26	26/26

L1: Deployment Constraints (expanded). *Strategy purity:* Our correctness proofs assume strategies are deterministic pure functions (Assumption 9). We enforce this through seeded randomness and canonical ordering, but strategies with external dependencies (e.g., data-dependent merging) require additional care.

Computational determinism: The convergence guarantee requires Assumption 10. In practice, this is achieved through containerised deployment, deterministic CUDA operations, or quantised representations (Section 7.1). Our single-GPU experiments cannot assess cross-hardware reproducibility; multi-node validation remains open.

Delta-state synchronisation: For billion-parameter models, transmitting the full OR-Set state is impractical. Delta-state CRDTs [2], transmitting only new (e, t, n) add entries and tombstoned tags, are essential for practical deployment at scale. Adaptation is straightforward—the OR-Set merge (Eq. 7) already decomposes into independent set unions, each of which admits incremental delta propagation—but has not yet been implemented or empirically evaluated in the current prototype.

Version vector scaling: Version vectors scale as $O(n)$ in the number of participating nodes. For the typical consortium scenario ($n < 100$), this is negligible. For $n > 1{,}000$, dotted version vectors [24] or interval-based clocks should replace the current implementation. The architecture is agnostic to the causal-ordering mechanism.

L2: Semantic Evaluation (expanded). The architecture does not address *semantic* convergence—while all replicas compute the same merged model, the quality depends on the underlying strategy. Our Tier 2 experiments validate CRDT property compliance (syntactic convergence) but do not evaluate downstream task performance. As Remark 16 establishes, the CRDT wrapper is transparent to the merge computation, so the extensive literature on strategy quality applies directly. Comprehensive benchmarking (e.g., MMLU, HellaSwag, domain-specific evaluations) is an important complement.

L3: Scalability (expanded). The system recomputes the merged model from the full contribution set on every resolve() call, incurring $O(k \cdot p)$ memory and $T_\sigma(k, p)$ computation. Incremental or hierarchical strategies maintaining CRDT compliance are needed for very large contribution sets.

The OR-Set's remove set R grows monotonically. Causal stability analysis [3] identifies tombstones observed by all replicas, which can be safely discarded. Garbage collection must be deferred until after resolve() has been executed and its output disseminated, ensuring

all replicas resolve against the same visible set before metadata is pruned. Empirical characterisation of tombstone growth rates and GC overhead under realistic workloads (frequent model retraction, long-running deployments) remains open; for the typical consortium scenario with $k < 100$ contributions and infrequent removals, metadata overhead is expected to be negligible.

L4: Security and Conflict Resolution (expanded). The OR-Set's add-wins policy means a concurrent add survives a concurrent remove. If removal represents discovery of a poisoned model, this could re-introduce harmful contributions. A remove-wins variant or explicit contribution validation (cryptographic attestation, reputation scoring) could address this.

The architecture does not currently include Byzantine fault tolerance. The OR-Set accepts all properly formatted contributions; a malicious participant could inject poisoned parameters. As discussed in Section 7.2 (L4), the two-layer architecture structurally enables a trust-as-CRDT extension: trust evidence (equivocation proofs, Merkle-root divergence, anomaly scores) would propagate through Layer 1 as a monotonic lattice, and a trust-gated merge at Layer 2 could provably exclude Byzantine contributions once the evidence lattice has converged. This offers a promising structural pathway toward Byzantine exclusion at known threat levels—all honest nodes would converge to identical trust states and therefore identical gated outputs—rather than the probabilistic tolerance offered by traditional robust aggregation. Integration with Byzantine-resilient aggregation [4] remains possible as a complementary defence.

Scale-dependent associativity: The observation that strategies failing on controlled tensors pass within tolerance on production-scale weights (Section 6.3) raises theoretical questions about the relationship between algebraic properties and numerical tolerance in high-dimensional spaces. Analysis of when associativity violations vanish at scale could inform design of "nearly associative" strategies.

F Per-Strategy Formal Analyses

The main text (Section 3) presents representative analyses for weight averaging and SLERP. We analyse the remaining major strategy families here.

TIES-Merging. TIES [33] operates through trimming, sign election, and disjoint merge. Sign election over sets is commutative (majority vote does not depend on enumeration order), but the binary merge operation is order-dependent because trimming on pairs versus the full set discards different entries [33]. Associativity fails because trimming thresholds depend on the set of vectors being merged—merging (a, b) first applies a different threshold than merging (b, c) first. Idempotency fails because trimming thresholds are recomputed, and the trim-then-merge pipeline on $\{a, a\}$ need not recover a.

DARE. DARE [37] applies a stochastic mask $m \sim \text{Bernoulli}(1-p)$ and rescales by $1/(1-p)$. All three properties fail: the stochastic mask produces different results on each invocation (violating commutativity and idempotency), rescaling factors compound under composition (violating associativity), and the mask differs per call (violating idempotency). In Phase 1 testing, stochastic strategies were evaluated without fixed seeds to reflect their default behaviour. The CRDT architecture (Phase 2) resolves this by deriving deterministic seeds from the Merkle root (Section 4.3).

Fisher-Weighted Merging. Fisher merging is commutative because summation is commutative [22]. However, associativity fails because intermediate Fisher information is lost during pairwise merging: the Fisher matrix of a merged model is not the sum of the constituent Fisher matrices. Idempotency holds: merging a model with itself using identical Fisher weights returns the original model.

The remaining strategies (21 of 26) follow similar patterns: stochastic strategies (evolutionary, genetic merge) fail all three properties; sparsification methods (model breadcrumbs, split–unlearn merge) fail idempotency; and all strategies involving normalisation or nonlinear composition fail associativity. Complete per-strategy results are in Tables 3 and 1.

G Data Flow Example

We illustrate the data flow with a two-node merge scenario. Consider nodes N_1 and N_2, each with initial states S_1 and S_2:

1. N_1 fine-tunes a base model and calls $\text{add}(S_1, \theta_1)$, producing state S_1' with updated add set, version vector, and Merkle hash.
2. N_2 independently fine-tunes and calls $\text{add}(S_2, \theta_2)$, producing S_2'.
3. When N_1 and N_2 synchronise (in either order), both compute $\text{merge}(S_1', S_2') = \text{merge}(S_2', S_1')$ by commutativity [29].
4. Both nodes now have identical visible sets: $\{\theta_1, \theta_2\}$.
5. Both nodes call $\text{resolve}(\cdot, \sigma, \cdot)$, sorting by hash, seeding randomness identically, and obtaining the same merged model θ^*.

For multi-party convergence with $k > 2$ nodes, associativity guarantees that the order of pairwise state merges does not affect the final state [28]. Whether node

N_3 merges first with N_1 or N_2, the final visible set—and therefore the resolved model—is identical once all states have been exchanged.

H Model Details

Table 5 lists the HuggingFace identifiers for all models used in Tier 2 experiments.

I Multi-Node Convergence Suite Results

All experiments use the `crdt-merge` library v0.9.4.

I.1 Multi-Node Convergence

Table 6 reports convergence results for 100 nodes across 20 independently randomised gossip orderings. Every ordering produces a bitwise-identical resolved model, confirming that the CRDTMergeState architecture achieves strong eventual consistency regardless of communication order.

I.2 Network Partition and Healing

One hundred nodes are split into 10 partitions (10 nodes each). Each partition gossips internally and converges to a distinct, consistent hash. After partition healing, all 100 nodes converge to a single bitwise-identical result. The final hash matches the multi-node convergence result, confirming deterministic SEC recovery.

I.3 Cross-Strategy Convergence Sweep

Table 8 verifies that all 26 merge strategies converge to a single canonical hash across 10 nodes. This confirms that the two-layer architecture provides strategy-independent convergence: every strategy, regardless of its algebraic properties, produces an identical resolved model on every node. Note that population-based strategies (`evolutionary_merge`, `genetic_merge`) incur substantially higher resolve times due to their internal search processes.

I.4 Scalability Benchmark

Table 9 measures how gossip and resolve times scale as the number of participating nodes increases from 2 to 50. Gossip time grows quadratically in the number of nodes (reflecting all-pairs state exchange), while per-call `merge()` cost remains constant in tensor size. As noted in Section 6.5, this prototype gossip protocol is designed for validation purposes; production deployments beyond $\sim$50 nodes would benefit from optimised dissemination protocols.

Table 5: HuggingFace model identifiers for Tier 2 evaluation.

Role	Identifier
GPT-2-XL (1.5 B parameters)	
Base	`openai-community/gpt2-xl`
Instruct	`nicholasKluge/Aira-2-1B5`
Domain	`lgaalves/gpt-2-xl_camel-ai-physics`
Wiki	`Clover-Hill/gpt2-xl-finetuned-wikitext103`
Mistral-7B-v0.1 (7.24 B parameters)	
Base	`mistralai/Mistral-7B-v0.1`
Instruct	`mistralai/Mistral-7B-Instruct-v0.2`
Hermes	`NousResearch/Nous-Hermes-2-Mistral-7B-DPO`
Zephyr	`HuggingFaceH4/zephyr-7b-beta`

Table 6: 100-node convergence across 20 random gossip orderings. Strategy: `slerp`; tensor: 512 × 512 (26,214,400 params); merges per ordering: 9,900.

Ordering	Gossip	Resolve	Max Diff	Status
1	503.1 ms	21,764 ms	0	PASS
2	523.6 ms	20,243 ms	0	PASS
3	457.4 ms	19,728 ms	0	PASS
4	446.7 ms	19,137 ms	0	PASS
5	472.1 ms	19,670 ms	0	PASS
6	482.7 ms	19,611 ms	0	PASS
7	541.7 ms	19,553 ms	0	PASS
8	456.4 ms	20,133 ms	0	PASS
9	465.5 ms	19,629 ms	0	PASS
10	445.1 ms	19,885 ms	0	PASS
11	459.5 ms	19,912 ms	0	PASS
12	479.0 ms	18,830 ms	0	PASS
13	624.7 ms	18,316 ms	0	PASS
14	430.5 ms	20,551 ms	0	PASS
15	481.5 ms	19,855 ms	0	PASS
16	612.1 ms	20,569 ms	0	PASS
17	486.4 ms	20,642 ms	0	PASS
18	562.9 ms	18,621 ms	0	PASS
19	460.8 ms	18,822 ms	0	PASS
20	464.4 ms	18,528 ms	0	PASS
Avg gossip:	492.8 ms	All orderings bitwise equal:		**YES**

Table 7: Network partition and healing: 100 nodes split into 10 isolated partitions, then healed. Each partition converges to a distinct hash during isolation; after healing, all nodes converge to a single bitwise-identical result matching the unpartitioned experiment (Table 6).

Metric	Value
Nodes / Partitions	100 / 10
Partition gossip time	4.6 ms
Distinct partition hashes	10/10
Healing time	556.6 ms
Post-healing convergence	100/100 nodes
Bitwise identical	YES
Final hash matches Table 6	YES

Table 8: Cross-strategy convergence: all 26 strategies on 10 nodes, 64 × 64 tensors. All strategies produce the same canonical hash, confirming strategy-independent convergence. Note: `evolutionary_merge` and `genetic_merge` exhibit resolve times of ∼24 h due to population-based search; all other strategies resolve in under 200 ms.

Strategy	Gossip	Resolve	Status
`ada_merging`	0.6 ms	200 ms	PASS
`adarank`	0.6 ms	102 ms	PASS
`dam`	0.7 ms	112 ms	PASS
`dare`	0.6 ms	80 ms	PASS
`dare_ties`	1.0 ms	79 ms	PASS
`della`	0.8 ms	122 ms	PASS
`dual_projection`	0.6 ms	26 ms	PASS
`emr`	0.6 ms	22 ms	PASS
`evolutionary_merge`	0.7 ms	86,400 ms	PASS
`fisher_merge`	0.6 ms	7 ms	PASS
`genetic_merge`	0.5 ms	87,960 ms	PASS
`led_merge`	0.6 ms	143 ms	PASS
`linear`	0.7 ms	11 ms	PASS
`model_breadcrumbs`	0.6 ms	22 ms	PASS
`negative_merge`	0.7 ms	8 ms	PASS
`regression_mean`	0.7 ms	8 ms	PASS
`representation_surgery`	0.7 ms	9 ms	PASS
`safe_merge`	0.6 ms	29 ms	PASS
`slerp`	0.6 ms	14 ms	PASS
`split_unlearn_merge`	0.6 ms	43 ms	PASS
`star`	0.8 ms	101 ms	PASS
`svd_knot_tying`	0.8 ms	154 ms	PASS
`task_arithmetic`	0.7 ms	8 ms	PASS
`ties`	0.6 ms	29 ms	PASS
`weight_average`	0.7 ms	8 ms	PASS
`wt_scope_alignment`	0.7 ms	18 ms	PASS

Table 9: Scalability: `slerp` on 64 × 64 tensors, 2–50 nodes. Gossip time scales as $O(n^2)$ (all-pairs merges); per-call `merge()` is $O(1)$ in tensor size.

Nodes	Params	Merges	Gossip	Resolve	Status
2	8,192	2	0.0 ms	0.8 ms	PASS
5	20,480	20	0.1 ms	3.7 ms	PASS
10	40,960	90	0.6 ms	16.8 ms	PASS
20	81,920	380	4.2 ms	74.7 ms	PASS
30	122,880	870	12.3 ms	163.9 ms	PASS
50	204,800	2,450	127.3 ms	445.2 ms	PASS